

MERN Stack & Full-Stack System Design Interview Bible

Production Internals, React 18, Node.js Event Loop, MongoDB Engineering & AI Architectures

1. React.js In-Depth & Modern Engineering

A. Virtual DOM, Reconciliation & React Fiber Architecture

- **Virtual DOM:** In-memory lightweight JSON representation of the real DOM tree.
- **React Fiber:** The re-implementation of React's core algorithm (React 16+). Fiber breaks rendering work into incremental units of work (workInProgress tree vs current tree). Enables pausing, prioritizing, and aborting work to ensure 60fps responsiveness.
- **Diffing Heuristics:** $O(N)$ diffing algorithm based on 2 assumptions: (1) Two elements of different types produce different trees, (2) Keys are stable and unique across sibling elements.

B. React Hooks Deep-Dive & Execution Mechanics

- **useState & useReducer:** Linked list of hook nodes on the Fiber node. Hook call order MUST remain identical across renders (never put hooks inside `if` conditionals or loops).
- **useEffect vs useLayoutEffect:** useEffect executes asynchronously AFTER paint (non-blocking). useLayoutEffect executes synchronously BEFORE paint (use for DOM measurement to prevent layout flicker).
- **useCallback vs useMemo:** useCallback(fn, deps) caches function instance. useMemo(() => val, deps) caches return value. Prevents unnecessary child component re-renders when passed as props.

```
// Custom Hook for Debouncing API Queries (e.g. Live Search)
import { useState, useEffect } from 'react';

export function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(handler); // Cleanup on rapid typing
  }, [value, delay]);

  return debouncedValue;
}
```

2. Node.js Architecture, Event Loop & Express

A. Node.js Event Loop 6 Phases

Node.js uses single-threaded event-driven architecture powered by **Libuv**.

NODE.JS EVENT LOOP PHASES

The 6 Event Loop Phases:

1. **Timers Phase:** Executes callbacks scheduled by `setTimeout()` and `setInterval()`.
2. **Pending Callbacks Phase:** Executes I/O callbacks deferred to the next loop iteration (e.g. TCP errors).
3. **Idle, Prepare Phase:** Internal Node.js usage.
4. **Poll Phase:** Retrieves new I/O events; executes I/O related callbacks (fs, network, db). Node will block here if queue is empty.
5. **Check Phase:** Executes callbacks invoked by `setImmediate()`.
6. **Close Callbacks Phase:** Executes close events (e.g. `socket.on('close')`).

Microtasks Priority: `process.nextTick()` and `Promise.then()` queues execute IMMEDIATELY after the current operation finishes, before moving to the next phase!

B. Streams & Backpressure

Streams handle data chunk-by-chunk without loading whole files into memory. Backpressure occurs when Writable stream cannot process data as fast as Readable stream produces it. Handle using `readable.pipe(writable)`.

C. Express Security & Auth Best Practices

- **JWT Security:** Short-lived Access Token (15 mins, in memory) + Long-lived Refresh Token (7 days, stored in `httpOnly`, `Secure`, `SameSite=Strict` Cookie). Prevents XSS & CSRF.
- **Security Headers & Middleware:** Helmet (headers), Express Rate Limit (DDoS protection), CORS whitelist, Express-mongo-sanitize (NoSQL injection prevention).

3. MongoDB Indexing, Aggregation & Database Engineering

A. Indexing Strategies & The ESR Rule

Indexes use B-Trees. Without indexes, MongoDB performs a full collection scan (COLLSCAN).

- **ESR Rule for Compound Indexes:** Equality fields first -> Sort fields second -> Range fields last.

Example: Query ``db.users.find({ status: 'ACTIVE', age: { $gte: 21 } }).sort({ createdAt: -1 })`` -> Index: ``{ status: 1, createdAt: -1, age: 1 }``.

B. MongoDB Aggregation Pipeline

```
// Aggregation Pipeline for Analytics Dashboard
db.orders.aggregate([
  { $match: { status: "COMPLETED", createdAt: { $gte: new Date("2026-01-01") } } },
  { $group: {
    _id: "$category",
    totalRevenue: { $sum: "$amount" },
    avgOrderValue: { $avg: "$amount" },
    totalOrders: { $sum: 1 }
  }},
  { $sort: { totalRevenue: -1 } },
  { $limit: 10 }
]);
```

4. Architectural Deep-Dive & Pitch for Your AI Projects

Interviewers love real system architecture details. Here is how to present your AI projects with full engineering maturity:

PROJECT ARCHITECTURAL SHOWCASE

1. K6Lab (AI-Powered Performance Testing & Diagnostics):

- **Problem Solved:** Manual load testing with K6 produces raw metrics that take hours to diagnose manually.
- **System Architecture:** Built Node.js backend runner spawned as child process streaming stdout/stderr metrics -> Server-Sent Events (SSE) streaming live throughput graphs to React client -> Automated LLM pipeline analyzing performance bottlenecks -> MongoDB Compound Indexed storage.
- **Key Metric Claim:** Reduced load-test metric analysis time from 45 minutes to 15 seconds.

2. Productivity AI Tool:

- **Problem Solved:** High LLM API latency & token cost inefficiency in interactive productivity suites.
- **System Architecture:** React + Zustand state store -> Express middleware with Redis caching layer (caching repeated prompt embeddings) -> Context trimming engine preventing prompt window inflation -> JWT auth with HttpOnly cookies.
- **Key Metric Claim:** Reduced OpenAI token cost by 35% using local Redis response caching.